DOCKER

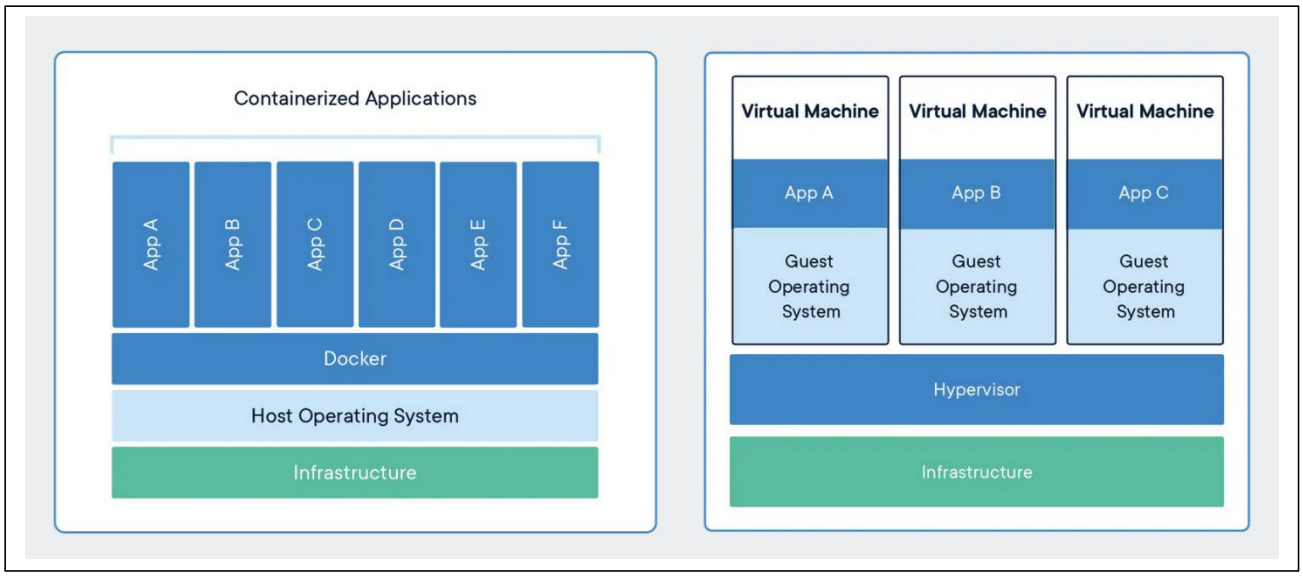
Varshitha Molabanti

What is a container ?

A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.

Ok, let me make it easy !!!

A container is a bundle of Application, Application libraries required to run your application and the minimum system dependencies.



Containers vs Virtual Machine

Containers and virtual machines are both technologies used to isolate applications and their dependencies, but they have some key differences:

1. Resource Utilization: Containers share the host operating system kernel, making them lighter and faster than VMs. VMs have a full-fledged OS and hypervisor, making them more resource-intensive.

2. Portability: Containers are designed to be portable and can run on any system with a compatible host operating system. VMs are less portable as they need a compatible hypervisor to run.

3. Security: VMs provide a higher level of security as each VM has its own operating system and can be isolated from the host and other VMs. Containers provide less isolation, as they share the host operating system.

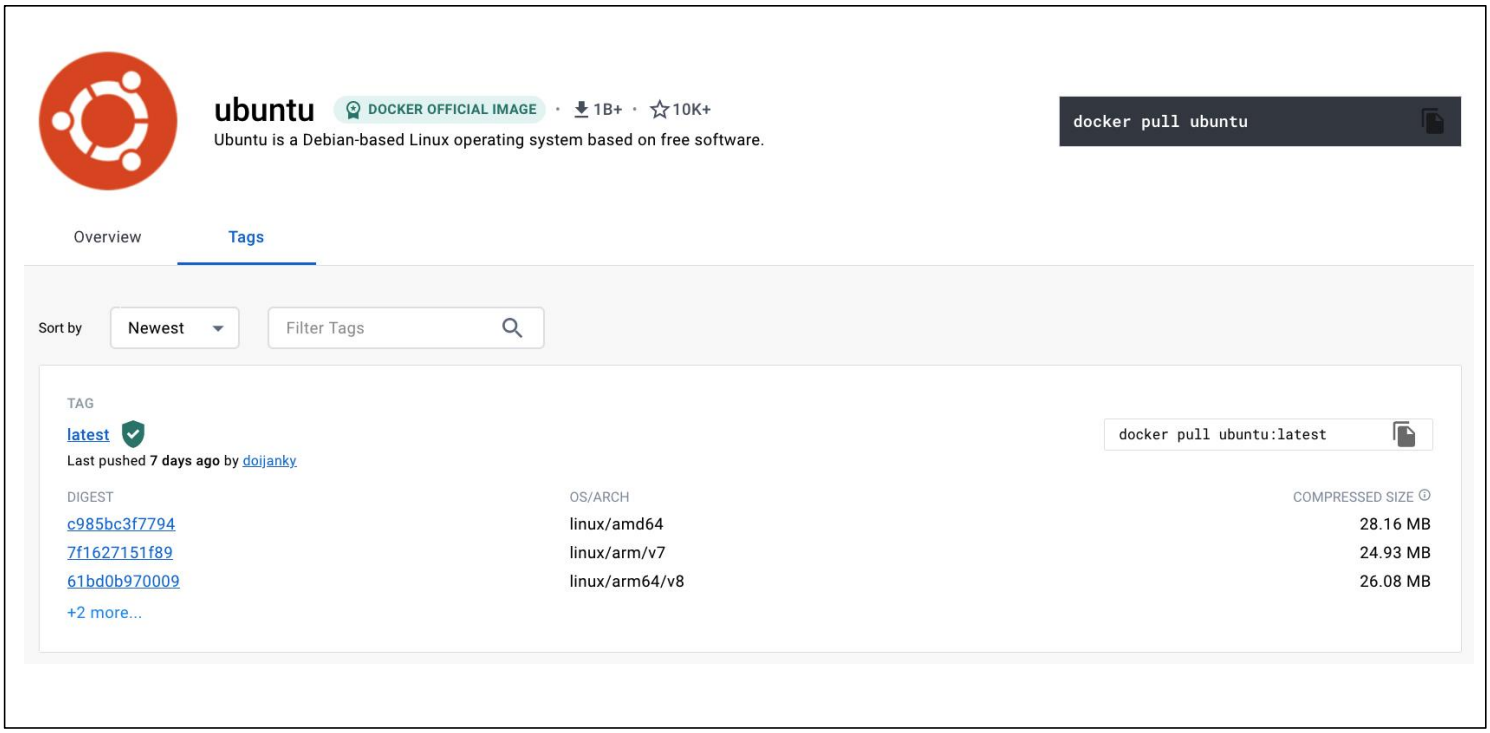
4.Management: Managing containers is typically easier than managing VMs, as containers are designed to be lightweight and fast-moving.

Why are containers light weight ?

Containers are lightweight because they use a technology called containerization, which allows them to share the host operating system's kernel and libraries, while still providing isolation for the application and its dependencies. This results in a smaller footprint compared to traditional virtual machines, as the containers do not need to include a full operating system. Additionally, Docker containers are designed to be minimal, only including what is necessary for the application to run, further reducing their size.

Let's try to understand this with an example:

Below is the screenshot of official ubuntu base image which you can use for your container. It's just ~ 22 MB, isn't it very small ? on a contrary if you look at official ubuntu VM image it will be close to ~ 2.3 GB. So the container base image is almost 100 times less than VM image.



To provide a better picture of files and folders that containers base images have and files and folders that containers use from host operating system (not 100 percent accurate -> varies from base image to base image). Refer below.

Files and Folders in containers base images

/bin: contains binary executable files, such as the ls, cp, and ps commands.

/sbin: contains system binary executable files, such as the init and shutdown commands.

/etc: contains configuration files for various system services.

/lib: contains library files that are used by the binary executables.

/usr: contains user-related files and utilities, such as applications, libraries, and documentation.

/var: contains variable data, such as log files, spool files, and temporary files.

/root: is the home directory of the root user.

Files and Folders that containers use from host operating system

The host's file system: Docker containers can access the host file system using bind mounts, which allow the container to read and write files in the host file system.

Networking stack: The host's networking stack is used to provide network connectivity to the container. Docker containers can be connected to the host's network directly or through a virtual network.

System calls: The host's kernel handles system calls from the container, which is how the container accesses the host's resources, such as CPU, memory, and I/O.

Namespaces: Docker containers use Linux namespaces to create isolated environments for the container's processes. Namespaces provide isolation for resources such as the file system, process ID, and network.

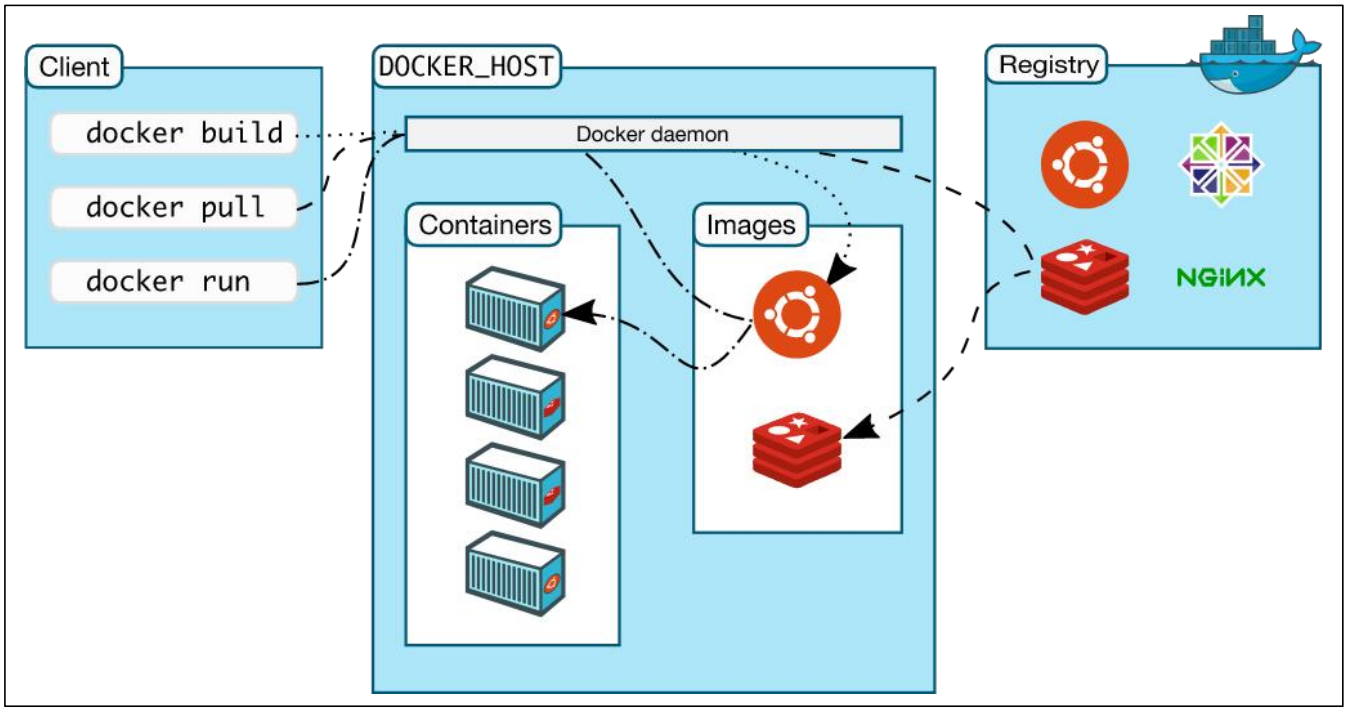
Control groups (cgroups): Docker containers use cgroups to limit and control the amount of resources, such as CPU, memory, and I/O, that a container can access.

What is Docker ?

Docker is a containerization platform that provides easy way to containerize your applications, which means, using Docker you can build container images, run the images to create containers and also push these containers to container regestries such as DockerHub, Quay.io and so on.

In simple words, you can understand as containerization is a concept or technology and Docker Implements Containerization.

Docker Architecture ?



The above picture, clearly indicates that Docker Deamon is brain of Docker. If Docker Deamon is killed, stops working for some reasons, Docker is brain dead :p (sarcasm intended).

Docker LifeCycle

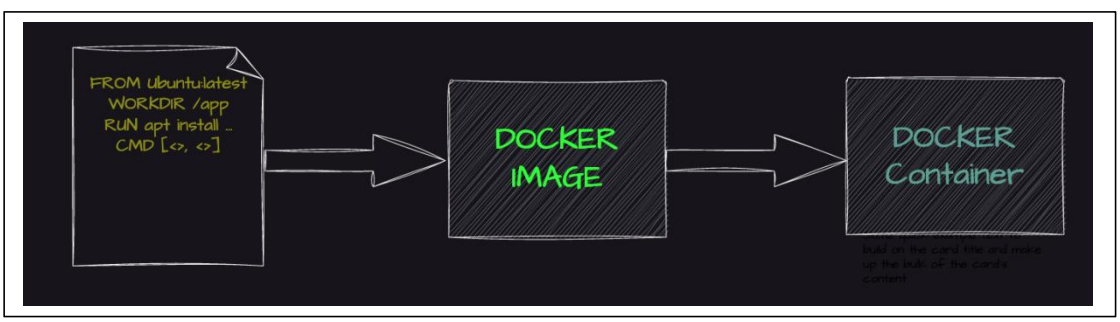
We can use the above Image as reference to understand the lifecycle of Docker.

There are three important things,

docker build -> builds docker images from Dockerfile

docker run -> runs container from docker images

docker push -> push the container image to public/private regestries to share the docker images.



Understanding the terminology (Inspired from Docker Docs)

Docker daemon

The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes. A daemon can also communicate with other daemons to manage Docker services.

Docker client

The Docker client (docker) is the primary way that many Docker users interact with Docker. When you use commands such as docker run, the client sends these commands to dockerd, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

Docker Desktop

Docker Desktop is an easy-to-install application for your Mac, Windows or Linux environment that enables you to build and share containerized applications and microservices. Docker Desktop includes the Docker daemon (dockerd), the Docker client (docker), Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper. For more information, see Docker Desktop.

Docker registries

A Docker registry stores Docker images. Docker Hub is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default. You can even run your own private registry.

When you use the docker pull or docker run commands, the required images are pulled from your configured registry. When you use the docker push command, your image is pushed to your configured registry. Docker objects

When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects. This section is a brief overview of some of those objects.

Dockerfile

Dockerfile is a file where you provide the steps to build your Docker Image.

Images

An image is a read-only template with instructions for creating a Docker container. Often, an image is based on another image, with some additional customization. For example, you may build an image which is based on the ubuntu image, but installs the Apache web server and your application, as well as the configuration details needed to make your application run.

You might create your own images or you might only use those created by others and published in a registry. To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it. Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt. This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

Installing Docker on Ubuntu EC2 (AWS)

1. Update System & Install Docker

```
sudo apt update
sudo apt install docker.io -y
```

2. Start Docker Daemon

Check if the Docker daemon is active:

```
sudo systemctl status docker
```

If not running, start it:

```
sudo systemctl start docker
```

3. Grant Docker Access to Current User

Add your user to the Docker group (replace ubuntu with your username):

```
sudo usermod -aG docker ubuntu
```

4. Verify Docker Installation

```
docker run hello-world
```

Running Your First Docker Project

1. Clone Example Repository

```
git clone https://github.com/iam-veeramalla/Docker-Zero-to-Hero
cd examples
```

2. Login to DockerHub

Create a DockerHub account: https://hub.docker.com/

```
docker login
```

3. Build Your First Docker Image

```
docker build -t <your-dockerhub-username>/my-first-docker-image:latest .
```

4. List Available Docker Images

```
docker images
```

5. Run the Docker Container

```
docker run -it <your-dockerhub-username>/my-first-docker-image
```

5. Push the Image to DockerHub

```
docker push <your-dockerhub-username>/my-first-docker-image
```

You've successfully installed Docker, built your first image, and pushed it to DockerHub!

You're now ready to explore Dockerfiles and advanced examples in the repository.

Dockerizing Your First Django App

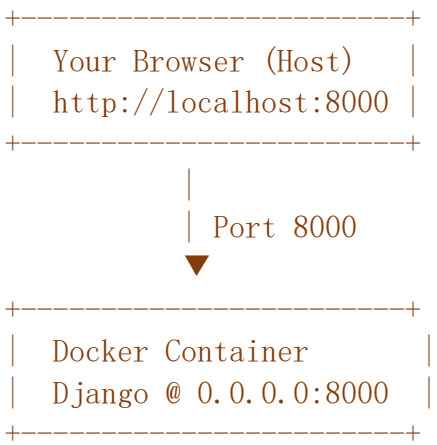
Step 1: Create a Simple Django Project
Step 2: Create requirements.txt
Step 3: Write the Dockerfile

```
# Use official Ubuntu image
FROM ubuntu:latest

# Set working directory inside the container
WORKDIR /app

# Install Python and pip
RUN apt-get update && apt-get install -y python3 python3-pip python3-venv

# Copy project files to /app in container
COPY . /app

# Install dependencies
RUN pip3 install --no-cache-dir -r requirements.txt

# Expose the port the app runs on (default Django port)

EXPOSE 8000

# Run Django development server
CMD ["python3", "manage.py", "runserver", "0.0.0.0:8000"]
```

Step 4: Build the Docker Image
docker build -t my-django-app .

Step 5: Run the Docker Container
docker run -p 8000:8000 my-django-app

Open your browser and go to:
http://localhost:8000
You should see the Django welcome screen!



What Is a Multi-Stage Docker Build?

Multi-stage builds allow you to use multiple FROM instructions in a single Dockerfile.
Each FROM starts a **new build stage**, and you can **copy artifacts** (e.g., built files) from earlier stages into the final image.

Why Use Multi-Stage Builds?

Problem with normal Dockerfile	Solution with Multi-Stage Build
Final image is large (includes build tools, temp files)	Only copy what's needed in the final image
Dependencies (e.g., compilers, SDKs) make image heavy	Use them in a temporary build stage
Complex build pipelines in one Dockerfile	Organize in logical stages (build → test → prod)

Project Structure

my-python-app/
├── app.py
├── requirements.txt
└── Dockerfile

Dockerfile (Multi-Stage Build)

```
# Stage 1: Build Stage
FROM python:3.10-slim as builder

WORKDIR /build

# Install dependencies
COPY requirements.txt .
RUN pip install --upgrade pip && \
    pip install --user -r requirements.txt

# Stage 2: Final Stage
FROM python:3.10-slim

WORKDIR /app

# Copy only necessary files
COPY --from=builder /root/.local /root/.local
COPY app.py .

# Set path for installed Python packages
ENV PATH=/root/.local/bin:$PATH

EXPOSE 5000

CMD ["python", "app.py"]
```

Advantages of Multi-Stage Builds

Smaller Images :Keeps only runtime code (no dev tools or compilers).
Security: Reduces attack surface (no unnecessary binaries).
Organized Builds: Cleaner, readable Dockerfiles.
Performance:Faster deploys and transfers.

Multi-Stage for Frontend + Backend
Dockerfile (React + Django)

```
### Stage 1: Build React App
FROM node:18 as frontend-build

WORKDIR /frontend
COPY frontend/ .
RUN npm install && npm run build

### Stage 2: Setup Backend (Django)
FROM python:3.10 as backend

WORKDIR /app

# Install Python dependencies
COPY backend/requirements.txt .
RUN pip install -r requirements.txt

# Copy backend code
COPY backend/ .

# Copy built frontend files into Django static directory
COPY --from=frontend-build /frontend/build /app/static/

EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Docker Volumes – Notes

Problem: Data Loss in Containers

By default, any data written inside a Docker container is **lost** when the container stops or is deleted.
This is because Docker containers are **ephemeral** — their file system is destroyed with the container.

Solution: Persisting Data in Docker

Docker provides **two ways** to persist data:

1. Volumes (Managed by Docker)

2. Bind Mounts (Using host machine directories)

1. Docker Volumes

What Are Volumes?

Volumes are **managed storage areas** on the host system used to **persist and share data** across containers.
They live outside the container's writable layer and are **not removed** when a container is deleted (unless explicitly told to).

Create a Docker Volume:
docker volume create myvolume

Docker Networking

Docker networking allows **communication between containers**, and between **containers and the host system**, even across multiple hosts.

Built-In Docker Networks

You can list existing Docker networks using:

```
docker network ls
```

NAME	DRIVER	Description
bridge	bridge	Default network for standalone containers
host	host	Shares host network directly
none	null	Isolated (no network)

1. Bridge Networking (Default)

Default network for containers on a single host.

Docker creates a virtual bridge (docker0) that acts like a router for containers.

Containers can communicate with each other and the host using IPs or aliases.

Custom Bridge Network

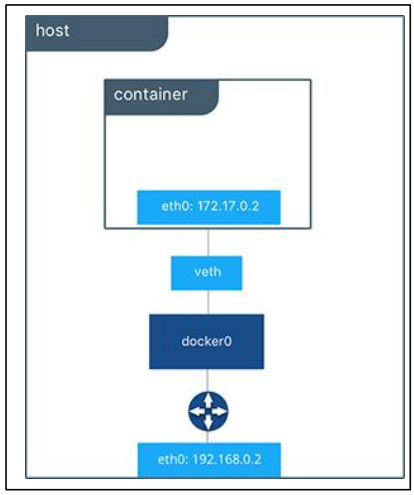
Create your own bridge:

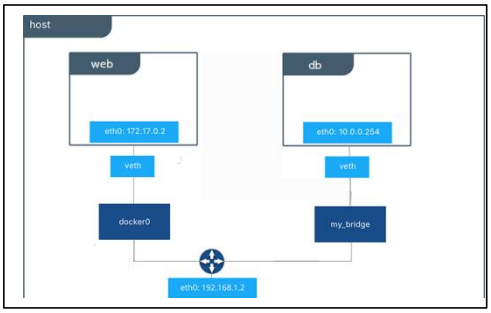
docker network create -d bridge my_bridge

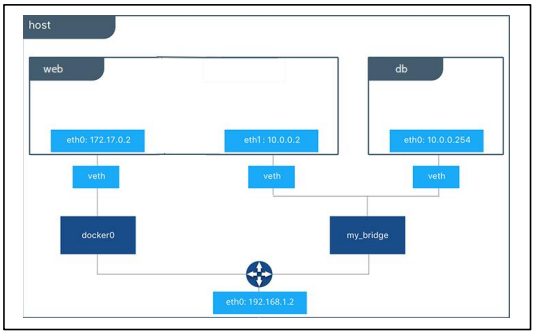
Run a container in it:

docker run -d --net=my_bridge --name db training/postgres

Containers in **different bridge networks** are isolated by default.
You can connect containers manually:
docker network connect my_bridge web







2. Host Networking

Container shares the host's network stack.

Container has no separate IP — it uses the host's IP and ports.

Useful for high-performance networking or when you need direct access to host services.

```
docker run --network="host" nginx
```

- **Less isolation** → higher **security risk**.
- Use only when necessary.

3. Overlay Networking

Used in Docker Swarm / multi-host scenarios.

Allows containers on different hosts to communicate as if they're on the same network.

Great for microservices spanning multiple nodes.

Requires Swarm mode and proper configuration

4. Macvlan Networking

Assigns a container its own MAC address.

Makes the container appear as a separate physical device on the network.

Useful when:

You want the container to have its own IP in LAN

Need to bypass NAT or firewall rules applied to the host

Driver	Scope	Use Case
bridge	Single host	Default, simple container-to-container comm.
host	Single host	Performance, direct host access
overlay	Multi-host	Swarm-based networking
macvlan	LAN access	Appears as physical device on network
none	None	Full isolation (no networking)

Docker Compose

What is Docker Compose?

Docker Compose is a **tool for defining and managing multi-container Docker applications** using a simple YAML configuration file.

Instead of running multiple `docker run` commands manually, Compose allows you to:

Define services (like app, db, redis) in one file.

Build and run everything with a single command: `docker-compose up`

Docker Compose File (`docker-compose.yml`)

This file contains all configuration needed to build and run multi-container applications.

```
version: "3.9"  # Optional, but good for compatibility

services:
  web:
    build: .
    ports:
      - "5000:5000"
  redis:
    image: "redis:alpine"
```

Installation (Linux)
sudo apt-get install docker-compose-plugin
docker compose version

In newer Docker versions, use docker compose (no hyphen). Older versions used docker-compose.

Task	Command
Start services	docker compose up
Start in background	docker compose up -d
Stop services	docker compose down
View logs	docker compose logs
View logs (live)	docker compose logs -f
Build images	docker compose build
List running containers	docker compose ps
Execute command in a container	docker compose exec service_name bash

Anatomy of docker-compose.yml

```yaml
version: "3.9"

services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    depends_on:
      - db

  db:
    image: postgres:13
    restart: always
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb

volumes:
  db_data:
```

Keyword	Purpose	Example
version	(Optional) Specifies Compose file format version	version: "3.9"
services	Top-level block to define services	services: web: ...
build	Builds image from Dockerfile	build: .
image	Specifies which image to use	image: nginx:alpine
container_name	Sets a fixed name for the container	container_name: my_web_app
command	Overrides the default container command	command: python manage.py runserver 0.0.0.0:8000
ports	Maps host ports to container ports	ports: ["8000:8000"]
volumes	Mounts host paths or named volumes	volumes: ["./app:/app", "db_data:/var/lib/postgresql/data"]
environment	Sets environment variables	environment: ["DEBUG=1", "SECRET_KEY=abc123"]
env_file	Loads environment variables from a file	env_file: .env
depends_on	Ensures one container starts before another	depends_on: ["db"]
networks	Attaches services to a network	networks: ["backend"]
restart	Configures container restart policy	restart: always

Example: Django + Postgres Compose Setup

Project Structure:

```
myproject/
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
└── mysite/
```

docker-compose.yml

```
version: "3.9"

services:
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    depends_on:
      - db

  db:
    image: postgres:13
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
```

Dockerfile

```
FROM python:3.10

WORKDIR /app
COPY . /app

RUN pip install --upgrade pip
RUN pip install -r requirements.txt
```

Cleaning Up

Stop and remove all services

docker compose down

Remove volumes too:

docker compose down -v

Deploying a MERN Stack Application using Docker Compose

1. Project Structure
Assume you have the following folders:

```
mern-docker/
├── backend/          # Node.js + Express app
│   ├── Dockerfile
│   └── ...
├── frontend/         # React.js app
│   ├── Dockerfile
│   └── ...
└── docker-compose.yml
```

2. Dockerfile for Backend (Node.js + Express)

```
# Use official Node.js image
FROM node:18

# Set working directory
WORKDIR /app

# Copy package files and install dependencies
COPY package*.json ./
RUN npm install

# Copy rest of the code
COPY . .

# Expose port
EXPOSE 5000

# Start app
CMD ["npm", "start"]
```

3. Dockerfile for Frontend (React)

```
# Build Phase
FROM node:18 as build

WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production Phase using Nginx
FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html

# Expose port 80
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

4.docker-compose.yml
At the root mern-docker/docker-compose.yml:

```
version: "3.8"

services:
  mongo:
    image: mongo
    container_name: mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  backend:
    build: ./backend
    container_name: backend
    ports:
      - "5000:5000"
    environment:
      - MONGO_URL=mongodb://mongo:27017/mydb
    depends_on:
      - mongo

  frontend:
    build: ./frontend
    container_name: frontend
    ports:
      - "3000:80"
    depends_on:
      - backend

volumes:
  mongo_data:
```

Commands to Build and Run

Navigate to the root project directory (mern-docker/) and run:

Build all images
docker-compose build

Start all containers
docker-compose up

To run in detached mode:
docker-compose up -d

To stop and remove containers:
docker-compose down

Access the Application

React App: http://localhost:3000

Express API: http://localhost:5000

MongoDB: Accessible inside the network as mongo:27017

Networking, Security, and Protocols

1. OSI Model (Open Systems Interconnection)

Layer	Name	Purpose	Examples
7	Application	Interface to applications	HTTP, FTP, SMTP
6	Presentation	Data translation, encryption, compression	SSL/TLS, JPEG
5	Session	Session management	NetBIOS, RPC
4	Transport	Reliable transmission, flow control	TCP, UDP
3	Network	Routing and addressing	IP, ICMP
2	Data Link	MAC addressing, error detection	Ethernet, PPP
1	Physical	Transmitting raw bits	Cables, Switches, NICs

Networking & Security Protocols

1. HTTP (HyperText Transfer Protocol)

Port: 80

Use: Transfers web page data (text, images, etc.) between a web browser (client) and a web server.

Why?: It allows users to browse websites and request data from servers.

Workflow:

1.You type http://example.com into your browser.

2.Browser (client) creates an HTTP request (e.g., GET /index.html).

3.Request is sent to the web server over port 80.

4.Server processes the request and responds with a status code (like 200 OK) and the requested content (like HTML).

5.Browser renders the web page for you to view.

All communication is in plain text – not secure!

2. HTTPS (HTTP Secure)

Port: 443

Use: Same as HTTP, but encrypted using TLS to protect privacy.

Why?: Prevents hackers from spying on your data (e.g., passwords, credit cards).

Workflow:

1.You type https://example.com in the browser.

2.Client sends a request to server to initiate a secure TLS handshake.

3.Server sends its SSL/TLS certificate (includes its public key).

4.Client verifies the certificate (checks against trusted Certificate Authorities).

5.If valid, the client creates a session key, encrypts it using the server's public key, and sends it back.

6.Server decrypts using its private key.

7.Both client and server now use the session key for secure communication.

8.Encrypted HTTP requests/responses (HTTPS) begin.

3. FTP (File Transfer Protocol)

Port: 21 (commands), 20 (data)

Use: Transfer files between a client and a remote server.

Why?: Used to upload/download files to websites or servers.

Workflow:

1.You open an FTP client (like FileZilla).

2.It connects to an FTP server at port 21.

3.You log in with a username/password.

4.The control connection is used to navigate folders and request actions.

5.A second connection (data channel on port 20) is used to transfer actual file data.

6.You upload/download files between local and remote systems.

Insecure – does not encrypt login or files.

SFTP (SSH File Transfer Protocol)

Port: 22

Use: Secure version of FTP, uses SSH encryption.

Why?: Safe file transfers over the internet.

Workflow:

1.SFTP client connects to the server on port 22.

2.User authenticates (password or SSH key).

3.SSH connection is established.

4.Encrypted file commands (upload/download) are sent.

Strong encryption via SSH.

5. SSH (Secure Shell)

Port: 22

Use: Secure login to remote servers (Linux/Unix).

Why?: Lets admins manage servers remotely with security.

Workflow:

1.User runs: ssh user@server.com

2.SSH client connects to server on port 22.

3.Server sends its public key.

4.Client verifies and sends encrypted credentials.

5.Server decrypts and logs you in.

6.get a remote terminal to run commands securely.

Used widely by system administrators.

DNS (Domain Name System)

Port: 53

Use: Converts domain names into IP addresses.

Why?: Easy for humans to remember `google.com` instead of `142.250.183.14`.

Workflow:

1.You type google.com in the browser.

2.Browser asks your local DNS resolver for the IP.

3.If not cached, resolver queries root DNS → TLD DNS → Authoritative DNS.

4.Final server returns IP address.

5.Resolver passes it to your browser.

Browser connects to the IP to load the website.

7. TLS/SSL (Transport Layer Security / Secure Socket Layer)

Use: Encrypts data in transit (used in HTTPS, IMAPS, etc.)

Why?: Prevents hackers from seeing or modifying data.

Workflow:

1.Client sends a "Client Hello" with supported encryption methods.

2.Server replies with "Server Hello" and certificate.

3.Client verifies certificate.

4.Key exchange occurs.

5,Secure session starts.

6.Data is encrypted using symmetric encryption.

TLS has replaced SSL (SSL is now deprecated).

8. SMTP (Simple Mail Transfer Protocol)

Port: 25 (basic), 587 (TLS), 465 (SSL)

Use: Sending emails.

Why?: Standard way for clients and mail servers to send email.

Workflow:

1.You hit "Send" in your email app.

2.App connects to SMTP server.

3.Authenticates (e.g., Gmail login).

4,Sends email (sender, receiver, subject, body).

4. Server relays email to recipient's mail server (e.g., via MX records).

9. IMAP/IMAPS (Internet Message Access Protocol)

Port: 143 (IMAP), 993 (IMAPS)

Use: Read/sync email without downloading.

Why?: Access email from multiple devices.

Workflow:

1.You open Gmail on your phone.

2.IMAP client connects to server.

3.Authenticates and fetches message headers.

4.You read a message – it's fetched on demand.

5.Email stays on server.

Best for cloud-based email access.

10. POP3/POP3S (Post Office Protocol)

Port: 110 (POP3), 995 (POP3S)

Use: Download email to one device.

Why?: Good for offline access (older usage).

Workflow:

1.Email client connects to POP3 server.

2.Authenticates.

3.Downloads inbox.

4.Optionally deletes from server.

Limited to single-device use.

11. SPF (Sender Policy Framework)

Use: Prevents spoofed sender email.

Why?: Email servers check if the sender is legitimate.

Workflow:

1.Mail server receives email from example.com.

2.Checks DNS SPF record of example.com.

3.If sender IP is not listed, mark as spam or reject.

12. DKIM (DomainKeys Identified Mail)

Use: Ensures email was not tampered.

Why?: Verifies email integrity and authenticity.

Workflow:

1.Sending mail server signs email with a private key.

2.Signature is included in email headers.

3.Receiving server gets public key from DNS.

4.Validates the signature.

13. DMARC (Domain-based Message Authentication, Reporting, and Conformance)

Use: Sets policy based on SPF and DKIM results.

Why?: Tells receiving servers how to handle failed SPF/DKIM.

Workflow:

1.Receiving mail server checks SPF and DKIM.

2.Compares domain alignment.

3.Applies policy from DNS (none, quarantine, reject).

4.Optionally sends report to domain owner.

14. Whitelisting, Greylisting, Blacklisting

Whitelisting: Always allow trusted IPs/domains.

Blacklisting: Block known malicious sources.

Greylisting:

Temporarily rejects unknown senders.

Legitimate servers retry → email accepted.

Spam bots usually don't retry → blocked.